



PROGRAMAÇÃO E ALGORITMOS

UMA JORNADA AO CORAÇÃO DA
SOLUÇÃO DE PROBLEMAS

Tiago Heineck

INSTITUTO FEDERAL CATARINENSE

Prefácio

Bem-vindo à aventura da programação, um campo fascinante que nos permite não apenas interagir com a tecnologia, mas também moldá-la e criar mundos. Este material didático foi concebido com o propósito de abrir as portas para aqueles que desejam se aventurar no universo da programação computacional, começando pelo aspecto mais fundamental: os algoritmos.

Endereçado a estudantes da disciplina de Algoritmos no curso de Ciência da Computação, este material procura estabelecer uma sólida compreensão dos princípios básicos da programação. A intenção é que, ao final deste curso, cada leitor não só consiga apreciar a elegância e a potência dos algoritmos, mas também esteja apto a utilizá-los como ferramentas para realizar suas próprias soluções de problemas.

Não importa se você está começando do zero ou buscando aprimorar seus conhecimentos, cada página da presente apostila foi estruturada para que o aprendizado seja progressivo, lógico e, acima de tudo, prático. Através do uso de pseudocódigo – um meio simplificado e linguagem agnóstica de descrever algoritmos –, este material ensinará como pensar de maneira computacional, modelar problemas e traduzir soluções para um formato que um computador possa executar.

À medida que você avança pelos capítulos, desafio você a abordar cada conceito e exercício não como uma tarefa a ser concluída, mas como uma peça de um quebra-cabeça mais amplo. Por trás de cada linha de código – ou pseudocódigo, neste caso – está uma decisão que tomamos para dar instruções precisas e resolver problemas cotidianos ou complexos.

Embora nossa viagem através deste material seja relativamente curta, as habilidades e conhecimentos que você adquirirá são fundamentais e perpétuos. Uma vez que você domine os conceitos básicos, estará equipado para seguir em direção a estudos mais avançados em programação, desenvolvimento de software e análise de sistemas.

Então, prepare-se para uma jornada intelectual estimulante. Cultive a curiosidade, pratique a paciência e desenvolva uma abordagem de resolução de problemas. Estou entusiasmado em guiá-lo por esse caminho de descoberta e espero que você encontre o mesmo prazer em aprender a programar que muitos antes de você descobriram.

Com entusiasmo pelos desafios à frente,

E sim, muito do conteúdo foi gerado por um robô de inteligência artificial criado para apoiar meus trabalhos! Sim, revisei! Importante ressaltar que não houve trabalho escravo de robôs, o GPT-4 foi remunerado, respeitando a CLTR (Consolidação das Leis Trabalhistas dos Robôs).

Prof. Tiago

Índice

Capítulo 1: O que é Programação?	6
Definição e objetivos da programação	7
Definição de Programação	7
Objetivos da Programação.....	7
A relação entre a programação e a resolução de problemas	8
Pensamento Lógico e Algorítmico	9
Identificação e Análise de Problemas.....	9
Desenvolvimento de Soluções Eficazes	9
Teste e Refinamento	10
Exemplos de aplicações no mundo real	10
O Problema: Gerenciamento de Fila de Espera em Unidades de Saúde	11
Capítulo 2: O que são Algoritmos?	14
Definição de algoritmos	14
Como os algoritmos são usados na programação	16
O Papel dos Algoritmos.....	16
O Processo de Transformação em Código	17
Exemplo Real de Uso de Algoritmo na Programação.....	17
Representações de algoritmo: fluxograma e pseudocódigo	18
Fluxograma.....	19
Características Principais do Fluxograma:	19
Exemplo de Fluxograma:	20
Pseudocódigo.....	20
Exemplo de Pseudocódigo:	21
Escolhendo a Ferramenta Certa.....	21
Capítulo 3: Pseudocódigo	22
Introdução ao pseudocódigo	23
Sintaxe básica e estrutura	25

Exemplo de pseudocódigo 1: "Hello World".....	28
Vamos praticar!.....	30
Vídeos para te ajudar:	34
Livros que te ajudarão:	35
Atividades:.....	35
Capítulo 4: Estruturas de Controle	36
IF e ELSE.....	37
Switch-Case.....	39
Laços de repetição: for, while, e do-while.	42
Atividade Problema:	44
Desafio: Sistema Automatizado de Irrigação de Jardins.....	46
Capítulo 5: Estruturas de Dados Simples	47
Variáveis e tipos de dados (int, float, string, boolean).	47
Vetores/Arrays.....	47
Exemplos de declaração e uso.	47
Exercício: Peça ao leitor para criar um pseudocódigo que armazene e imprima uma lista de compras.	47
Capítulo 6: Funções e Procedimentos	48
Definição de funções.....	48
Vantagens do uso de funções em algoritmos.....	48
Exemplo: função para calcular o quadrado de um número.....	48
Exercício: Desafie o leitor a criar uma função que calcule a média de três números.....	48
Conclusão	49
Resumo do que foi discutido.....	49
A importância de continuar praticando e estudando algoritmos e programação.....	49
Próximos passos recomendados.	49
Referências	50
Desafios e Exercícios Resolvidos	51

Capítulo 1: O que é Programação?

Seja bem-vindo ao canteiro de obras do pensamento lógico e da criatividade! A programação é o processo pelo qual erguemos as majestosas estruturas do mundo digital. Neste capítulo, iniciaremos nossa jornada compreendendo como os blocos de código se encaixam para criar as aplicações que transformam nosso cotidiano.

Imagine-se como um engenheiro ou arquiteto definindo os planos para uma nova construção. Antes de levantar a primeira parede, você precisa de um projeto detalhado que descreva cada aspecto da estrutura - desde a fundação até o telhado. Esse projeto é o seu roteiro para a realização da obra.

Na programação, esse roteiro toma a forma de um algoritmo. Assim como o arquiteto usa seu projeto para guiar a equipe na construção da casa, pedra por pedra, com o algoritmo guiamos o computador, instrução por instrução, para montar a estrutura lógica que constituirá nosso software. Ao longo deste capítulo, vamos aprender os elementos básicos que compõem o projeto de um programa de computador. Descobriremos o que são as linguagens de programação e como elas servem como as ferramentas – martelos, chaves de fenda, níveis – necessárias para transformar nosso plano em realidade.

Vamos vestir nossos capacetes de segurança e calçar nossas botas de trabalho, pois vamos adentrar no canteiro de obras da programação. Aqui, você se familiarizará com o uso de variáveis, que são como os blocos básicos de construção, as estruturas de controle de fluxo, que garantem que a sequência das tarefas seja realizada da maneira correta, e as funções, que são como se fossem pequenas equipes especializadas dentro de nossa construção, cada uma responsável por uma tarefa distinta.

Está na hora de pegar as ferramentas, conferir os planos e dar início à edificação do seu primeiro programa. Que tal construir algo que perdure no testemunho do tempo e que facilite a vida das pessoas, assim como uma bem planejada e executada construção faz no mundo real? Mãos à obra! Com essa analogia, espero ter criado uma visão tangível e inspiradora do que significa programar, e instigado a curiosidade e sua empolgação para o que vem a seguir. Construir software é mesmo

como construir estruturas duradouras, peça por peça, linha por linha de código. Vamos começar a construir?

Definição e objetivos da programação.

Quando paramos para observar uma paisagem urbana com seus arranha-céus, pontes e casas, o que vemos são os frutos concretos do planejamento e do trabalho de muitos profissionais. A programação, em seu âmago, é muito similar a essa construção civil, mas em vez de aço e concreto, usamos linhas de código e lógica para erguer estruturas virtuais dentro do mundo digital.

Definição de Programação

Programação é o processo de design, escrita, teste e manutenção de um programa de computador. O código escrito pelos programadores é o conjunto de instruções que o computador segue para realizar tarefas complexas, das mais simples às mais elaboradas. Essas instruções são escritas em uma linguagem de programação - um sistema de notação que combina comandos, símbolos e palavras-chave compreensíveis tanto para as máquinas quanto para os humanos com o conhecimento necessário.

Como o arquiteto que utiliza um conjunto de símbolos e regras para desenhar um blueprint, assim o programador utiliza a linguagem de programação para esboçar o algoritmo que servirá como esqueleto para o programa final. Este "blueprint digital" detalha cada operação, condicional, loop e interação que o computador deve seguir para executar a tarefa desejada.

Objetivos da Programação

Os objetivos da programação são tão variados quanto os tipos de estruturas que um engenheiro pode querer construir. Aqui estão alguns dos principais objetivos que motivam os programadores a escrever código:

1. **Resolver Problemas:** Tal como um engenheiro resolve problemas de infraestrutura, um programador resolve problemas digitais. Eles criam programas que facilitam atividades, otimizam processos e oferecem soluções inovadoras para necessidades do nosso dia-a-dia.
2. **Automatizar Tarefas:** Assim como as máquinas automatizam a mistura do concreto ou a montagem de partes de uma estrutura, os programas de computador automatizam tarefas repetitivas, reduzindo a necessidade de intervenção humana e aumentando a eficiência.
3. **Facilitar a Comunicação:** Programas de computador também podem ser vistos como pontes de comunicação – eles conectam pessoas, sistemas e dispositivos, permitindo uma interação fluida entre diferentes plataformas e culturas, assim como uma via expressa conecta bairros e cidades.
4. **Promover Inovação e Criatividade:** Da mesma forma que a arquitetura moderna permite a criação de edifícios com formas e funções jamais vistas antes, a programação oferece a oportunidade de explorar ideias novas e revolucionárias, que podem mudar a maneira como vivemos e pensamos.

Cada linha de código é um tijolo, cada função é um pilar e cada programa completo é um edifício que habitamos no espaço digital. Os programadores são os construtores deste mundo invisível, mas imensamente tangível em sua influência no mundo visível.

Nesta subseção, vamos escavar as fundações da programação para entender melhor o terreno em que trabalhamos. Vamos explorar os conceitos cruciais que formam a base de todos os programas e começar a visualizar as estruturas imponentes que podemos criar com o nosso conhecimento e habilidade.

A relação entre a programação e a resolução de problemas.

Como um engenheiro diante de um terreno em branco, o programador encara o desafio de transformar um espaço vazio em algo funcional e útil. No coração de toda programação repousa a essência de resolver problemas. Ao programar, somos como detetives, decifrando pistas e

elaborando estratégias para encontrar as soluções mais eficientes para os mistérios que se apresentam sob a forma de desafios computacionais.

Pensamento Lógico e Algorítmico

Para abordar a relação entre programação e resolução de problemas, é essencial desenvolver um pensamento lógico e algorítmico. Assim como um arquiteto divide um projeto grandioso em partes gerenciáveis, um programador decompõe um problema complexo em segmentos menores e mais fáceis de lidar. Este processo é conhecido como 'divisão e conquista' e é um dos pilares da lógica de programação.

Identificação e Análise de Problemas

Antes de escrever uma única linha de código, devemos entender profundamente o problema que estamos tentando solucionar. Tal como um arquiteto investiga o propósito de um edifício e as demandas do terreno, um programador deve fazer perguntas fundamentais:

Qual é a natureza do problema?

Quais são os objetivos a serem alcançados?

Quais dados são necessários para construir a solução?

Como podemos medir o sucesso da nossa solução?

Desenvolvimento de Soluções Eficazes

Uma vez identificado e analisado o problema, é hora de construir uma solução eficaz. O programador começa a esboçar algoritmos – os planos detalhados para cada parte da solução. Esses algoritmos são então transformados em código, criando programas que executam as tarefas desejadas.

Ao programar, não estamos simplesmente criando um conjunto aleatório de instruções; estamos organizando dados, operações e lógica em uma estrutura que não só resolve o problema como o faz de maneira eficiente e elegante. Buscamos construir uma solução que seja como um prédio bem projetado: não apenas funcional e robusto, mas também fácil de manter e expandir no futuro.

Teste e Refinamento

A construção não termina quando o último tijolo é colocado. Da mesma forma, a programação não termina quando o último ponto-e-vírgula é digitado. Testamos e refinamos o nosso código, assim como um engenheiro avalia a integridade de uma ponte. Este processo garante que a solução atenda a todas as expectativas e performe bem sob várias condições.

Ao final, a habilidade de programar transcende a mera codificação; é sobre desenvolver uma mentalidade que ativamente busca soluções, uma que antecipa problemas e responde com invenções inovadoras.

Exemplos de aplicações no mundo real.

No vasto terreno da vida cotidiana, nos deparamos com desafios que vão desde organizar uma agenda até otimizar o tráfego em uma cidade movimentada. A programação se destaca como a ferramenta definitiva para desbravar esse terreno e construir soluções inovadoras. Vamos ilustrar como um problema do mundo real pode ser abordado e resolvido com a ajuda de um algoritmo.

O Problema: Gerenciamento de Fila de Espera em Unidades de Saúde

Considere as unidades de saúde, onde os pacientes chegam a todo momento buscando atendimento. O desafio é organizar essa chegada de forma a minimizar o tempo de espera, garantir um atendimento justo e melhorar a experiência geral do paciente.

O Objetivo

O objetivo é projetar um sistema que gerencie eficientemente a fila de espera, atendendo a critérios como urgência do atendimento, horário de chegada e disponibilidade dos profissionais de saúde.

A Solução Proposta: Algoritmo de Gerenciamento de Fila

Para resolver esse problema, podemos desenvolver um algoritmo que, ao ser implementado em um programa, atenderá aos objetivos. Aqui está uma descrição de alto nível do algoritmo proposto:

1. **Coleta de Dados:** Quando um paciente chega, o sistema coleta informações básicas, incluindo o motivo da visita e o nível de urgência, que pode ser determinado por um questionário ou triagem inicial.
2. **Categorização:** Com base nos dados obtidos, o algoritmo classifica o paciente em uma das diversas filas, ordenadas por prioridade. Por exemplo, casos de emergência têm a mais alta prioridade, seguidos por casos urgentes, não urgentes e, por fim, consultas rotineiras.
3. **Alocação de Recursos:** O sistema monitora a disponibilidade dos profissionais de saúde e atribui pacientes da fila correspondente a eles assim que ficam disponíveis.
4. **Notificação:** O paciente recebe uma notificação sobre o tempo estimado de espera e é alertado quando está próximo de ser atendido, permitindo que ele não fique preso em uma sala de espera.

5. **Ajustes Dinâmicos:** O algoritmo ajusta a fila em tempo real conforme as circunstâncias mudam, como a chegada de um caso de emergência ou uma mudança na disponibilidade dos profissionais.

Implementação de Código (Pseudo-Código)

```
função gerenciarFila(paciente):
    dadosPaciente = coletarDados(paciente)
    prioridade = determinarPrioridade(dadosPaciente)

    filaPrioridade = adicionarAFila(paciente, prioridade)
    atualizarTempoEstimado(filaPrioridade)

    se profissionalDisponível(prioridade):
        pacienteAtual = proximoPaciente(filaPrioridade)
        alocarProfissional(pacienteAtual)

    notificarPaciente(filaPrioridade)
    realizarAjustesDinamicos(filaPrioridade)

quando pacienteChega(paciente):
    gerenciarFila(paciente)

quando profissionalFicaDisponível(profissional):
    para cada prioridade em prioridades:
        se filaPrioridade nao estiver vazia:
            alocarProfissional(profissional, proximoPaciente(prioridade))
            quebrar loop
```

Este algoritmo é um esboço simplificado, mas ilustra como um sistema de gerenciamento de fila de espera poderia funcionar numa unidade de saúde. Implementações reais envolveriam tratamento de exceções, integração com outros sistemas hospitalares e questões de privacidade e segurança dos dados dos pacientes.

Esse exemplo mostra a relevância da programação no desenvolvimento de soluções pragmáticas para problemas complexos. Com a aplicação de um algoritmo bem pensado, é possível não apenas

melhorar uma experiência específica, mas também contribuir para um sistema de saúde mais eficiente e humano.

Capítulo 2: O que são Algoritmos?

Bem-vindo ao Capítulo 2 da nossa jornada pelo fascinante mundo da programação! Aqui, vamos mergulhar no coração da computação - os algoritmos - explorando o que são, como são fundamentais para a programação e as diversas formas pelas quais podem ser representados e compreendidos.

Imagine que você está diante de um grande labirinto, onde seu objetivo é encontrar o caminho até o tesouro escondido. Nesse cenário, o algoritmo seria o seu mapa, uma série de instruções detalhadas que, se seguidas corretamente, garantiriam que você chegasse ao seu destino com sucesso, evitando becos sem saída e obstáculos inesperados.

Na programação, os algoritmos desempenham um papel semelhante: são a espinha dorsal por trás de cada linha de código, cada aplicativo e cada função digital que usamos em nossa vida diária. Sem eles, os computadores seriam meras caixas vazias, incapazes de realizar as tarefas incrivelmente complexas que desempenham hoje.

Neste capítulo, você aprenderá a definição formal de algoritmos e como eles são a chave para desenvolver programas eficazes. Também vamos desvendar as formas pelas quais os algoritmos podem ser representados - desde o abstrato fluxograma ao mais concreto pseudocódigo - cada um com seu próprio papel em tornar a complexidade acessível e o difícil, simples.

Prepare-se para embarcar nesta aventura de resolver enigmas lógicos e construir a base sólida de seus conhecimentos em programação. A frente nos aguarda a essência do pensamento computacional: a arte e a ciência dos algoritmos!

Definição de algoritmos.

Antes de mergulharmos na prática da programação, é essencial compreender o que são algoritmos. Como mencionado anteriormente, podemos pensar em algoritmos como mapas ou receitas, mas o que isso realmente significa no contexto da computação?

Algoritmo, em sua essência, é uma sequência clara e bem definida de instruções ou regras que são projetadas para realizar uma tarefa específica ou resolver um determinado problema. Um algoritmo é como um guia passo a passo que, se seguido à risca, leva a um resultado esperado, assim como uma receita culinária pode guiar alguém no processo de cozinhar uma refeição deliciosa.

Ao seguir um algoritmo, não importa quem o execute ou quantas vezes - se as instruções forem precisas e o contexto permanecer o mesmo, o resultado final será consistentemente o mesmo. Isso é crucial na programação, onde buscamos resultados previsíveis e confiáveis.

Características importantes de um algoritmo incluem:

- **Precisão:** As instruções devem ser específicas e inequívocas.
- **Finitude:** Deve existir um número finito de passos.
- **Efetividade:** Cada passo deve ser suficientemente básico para, em teoria, ser realizado manualmente.
- **Entradas:** Deve receber dados de entrada para processamento (embora nem sempre seja necessário).
- **Saídas:** Deve produzir um resultado ou saída, que é a finalidade do algoritmo.
- **Generalidade:** Comumente, o algoritmo deve ser aplicável a um conjunto de problemas similares, não apenas a uma instância ou situação única.

Com a definição de algoritmo bem estabelecida, podemos apreciar como eles são o alicerce da programação. Um bom algoritmo não só resolve o problema proposto como também o faz de maneira eficiente, utilizando o mínimo de recursos necessários - seja tempo de processamento ou memória.

Para ilustrar, vamos pensar em um algoritmo simples do nosso cotidiano: atravessar a rua.

Algoritmo "Atravessar a Rua"

1. Chegar à faixa de pedestres
2. Olhar para a esquerda

3. Se não houver veículos aproximando, ir para o passo 5
4. Se houver veículos, esperar até a via estar livre e voltar ao passo 2
5. Atravessar a faixa de pedestres
6. Chegar ao outro lado da rua

Este exemplo demonstra a forma como um algoritmo toma decisões baseadas em condições e executa ações com base nessas decisões. Através da programação, essa mesma lógica é transformada em código para operar em um contexto digital, com "decisões" sendo tomadas pelo computador a cada momento.

Na próxima seção, exploraremos como os algoritmos são usados na programação e como eles formam a base do desenvolvimento de software.

Como os algoritmos são usados na programação.

Agora que entendemos o que são algoritmos, é hora de explorar seu papel fundamental na programação. Assim como um diretor de cinema não começa a filmar sem um roteiro detalhado, um programador não começa a escrever código sem um algoritmo bem planejado.

O Papel dos Algoritmos

Algoritmos servem como o núcleo lógico por trás de todas as operações que um computador executa. Desde o simples ato de ordenar uma lista de números até o complexo processo de compressão de dados ou renderização de gráficos 3D, os algoritmos oferecem o roteiro para cada ação realizada.

O Processo de Transformação em Código

Transformar um algoritmo em um programa funcional envolve vários passos:

1. **Definição do Problema:** Primeiro, o problema precisa ser entendido claramente. Isso pode parecer simples, mas uma compreensão abrangente é essencial para criar um algoritmo eficaz.
2. **Planejamento do Algoritmo:** Aqui, o programador pensa na lógica necessária para resolver o problema, detalhando cada passo intermédio.
3. **Codificação:** Nesta etapa, o algoritmo é traduzido para uma linguagem de programação específica, convertendo a lógica em código que o computador pode executar.
4. **Testes e Depuração:** Após a codificação, o programa é testado para garantir que todos os casos sejam cobertos e que não haja erros ou "bugs".
5. **Otimização:** Dependendo dos resultados dos testes e da eficiência do programa, o algoritmo pode ser ajustado para melhorar o desempenho ou reduzir o uso de recursos.
6. **Manutenção:** A continuidade do ciclo de vida de um programa frequentemente envolve revisões e alteração de algoritmos para acomodar novos requisitos ou corrigir problemas identificados.

Exemplo Real de Uso de Algoritmo na Programação

Pense, por exemplo, em um aplicativo de lista de tarefas. O desafio é permitir que o usuário adicione, remova e ordene essas tarefas. Por trás dessas funcionalidades simples, os algoritmos estão trabalhando silenciosamente:

1. **Adicionar Tarefa:** Um algoritmo determina onde e como a nova tarefa é armazenada na estrutura de dados.

2. **Remover Tarefa:** Outro algoritmo é acionado para localizar e excluir a tarefa da lista, garantindo que todas as referências sejam devidamente atualizadas.
3. **Ordenar Tarefas:** Algoritmos de ordenação, como 'bubble sort' ou 'quick sort', entram em cena para organizar as tarefas com base em critérios como prioridade ou prazo.

```
# Exemplo simplificado de um algoritmo de ordenação em Python
def sort_tasks(tasks):
    n = len(tasks)
    for i in range(n):
        for j in range(0, n-i-1):
            if tasks[j]['priority'] > tasks[j+1]['priority']:
                tasks[j], tasks[j+1] = tasks[j+1], tasks[j]
    return tasks

tasks = [{'task': 'Comprar leite', 'priority': 2}, {'task': 'Enviar email',
'priority': 1}]
sorted_tasks = sort_tasks(tasks)
print(sorted_tasks)
```

Este trecho de código representa um algoritmo de ordenação simples que organiza uma lista de tarefas com base na prioridade.

Algoritmos são, portanto, o cerne de todos os sistemas de software. A eficiência, confiabilidade e usabilidade de um programa tipicamente dependem da qualidade dos algoritmos implementados. À medida que a sua aventura de programação se desdobra, lembrar-se de que cada linha de código começa com a lógica fundamental de um algoritmo irá guiá-lo a escrever programas não apenas que funcionam, mas que brilham em performance e inovação.

Na próxima seção, nos aprofundaremos nas representações de algoritmo mais comuns, entrando no mundo dos fluxogramas e pseudocódigos.

Representações de algoritmo: fluxograma e pseudocódigo.

Após compreendermos o que são algoritmos e como eles são usados na programação, vamos nos familiarizar com duas ferramentas essenciais para a representação de algoritmos: o fluxograma e

o pseudocódigo. Estas representações permitem aos desenvolvedores visualizar a lógica e a estrutura de um algoritmo antes de transformá-lo em código

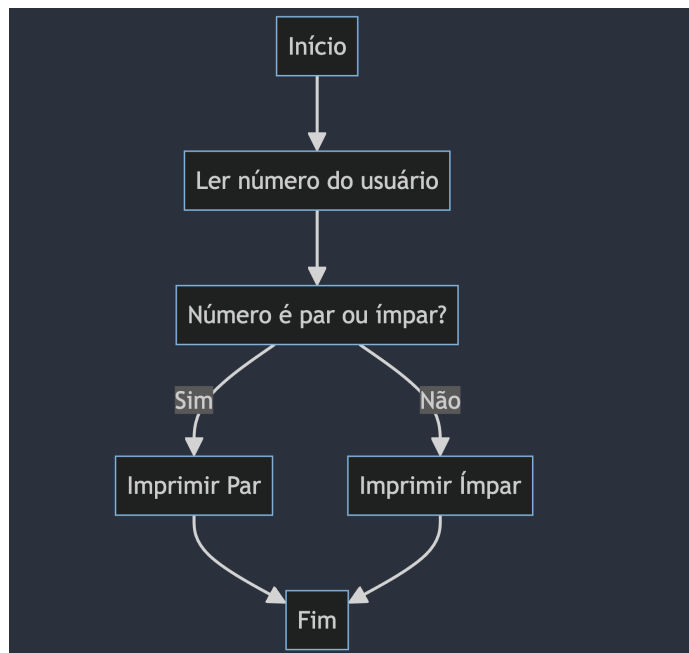
Fluxograma

O fluxograma é uma representação gráfica dos passos de um algoritmo, utilizando símbolos padronizados para diferentes tipos de ações e decisões. Com um olhar sobre um fluxograma, você pode rapidamente compreender o fluxo do algoritmo, mesmo sem conhecimento detalhado da sintaxe de uma linguagem de programação específica.

Características Principais do Fluxograma:

- **Símbolos:** Formas geométricas padronizadas como retângulos (para operações), losangos (para decisões), e ovais (para o início e fim).
- **Fluxo:** As setas conectam os símbolos, indicando a direção em que o algoritmo se move.
- **Clareza:** Fornece uma visão geral intuitiva do processo e ajuda a identificar possíveis pontos de melhoria ou erros na lógica.

Exemplo de Fluxograma:



O fluxograma acima demonstra um processo simples para verificar se um número é par ou ímpar.

Pseudocódigo

Pseudocódigo é uma descrição de alto nível de um algoritmo, escrita em linguagem humana que imita a sintaxe de programação sem estar vinculada a uma linguagem específica. O pseudocódigo permite focar na lógica do algoritmo, sem se preocupar com as regras sintáticas e a estrutura de uma linguagem de programação.

Benefícios do Pseudocódigo:

- **Independência de Linguagem:** Pode ser compreendido por pessoas com conhecimento de diferentes linguagens de programação.
- **Foco na Lógica:** Permite que os programadores esbocem e refaçam a lógica do algoritmo rapidamente.

- **Preparação para Codificação:** Facilita a transição para o código, servindo como um rascunho detalhado da implementação.

Exemplo de Pseudocódigo:

```
ALGORITMO VerificarParidade
INÍCIO
  Leia número
  SE número MOD 2 = 0 ENTÃO
    Imprima "Par"
  SENÃO
    Imprima "Ímpar"
  FIM SE
FIM
```

O pseudocódigo apresentado acima descreve o mesmo processo do fluxograma para determinar se um número é par ou ímpar.

Escolhendo a Ferramenta Certa

A escolha entre fluxograma e pseudocódigo pode depender das preferências pessoais do programador ou das necessidades do projeto. Alguns acham que fluxogramas são melhores para processos que envolvem várias decisões e condições, enquanto pseudocódigos podem ser preferidos para expressar algoritmos mais lineares ou detalhar estruturas complexas de dados.

Combinar ambas as representações oferece uma compreensão robusta do algoritmo - o fluxograma fornece uma visão geral visual, enquanto o pseudocódigo prepara o terreno para a codificação detalhada.

Com a compreensão dessas duas poderosas ferramentas de representação, os desenvolvedores estão melhor equipados para projetar, comunicar e implementar seus algoritmos com clareza e precisão.

Capítulo 3: Pseudocódigo

Queridos estudantes e entusiastas da Ciência da Computação, sejam muito bem-vindos ao capítulo que considero um dos mais empolgantes em nossa aventura pelo mundo da programação! Neste capítulo, vamos falar sobre o pseudocódigo, uma ferramenta que vai ajudar vocês a dar os primeiros passos no processo de traduzir ideias em código de maneira lúdica e descomplicada.

Imagine que vocês estão construindo uma casa. Antes de erguer as paredes e pintar as fachadas, é preciso ter uma planta, um esboço de como tudo vai ficar. No universo da programação, o pseudocódigo é como essa planta arquitetônica: um modo de expressar as intenções da lógica de um programa de forma simples e de fácil compreensão, sem a necessidade de se preocupar com a complexidade e as regras específicas de uma linguagem de programação.

No tópico de "Introdução ao pseudocódigo", vamos mergulhar na essência do porquê e de como ele é usado por programadores ao redor do mundo, independente da linguagem com a qual trabalhem. Este versátil instrumento é o primeiro passo para organizarmos nossas ideias de forma estruturada, enfatizando a lógica por trás do programa que desejamos construir.

Avançando para "Sintaxe básica e estrutura", vamos explorar os componentes simples do pseudocódigo, como "se-então-senão" para tomadas de decisão, "enquanto" e "para" em laços de repetição, e a forma de declaração de variáveis. Apesar de não ser uma linguagem formal com regras rigorosas, o pseudocódigo tem sua estrutura lógica, que garante que qualquer pessoa com conhecimento básico possa entender o que você deseja programar. Vou mostrar como transformamos nosso pensamento em uma sequência lógica de passos, que mais tarde poderão ser facilmente traduzidos para uma linguagem de programação real.

E por fim, nada melhor do que um exemplo prático! Em "Exemplo de pseudocódigo 1: 'Hello World'", vamos desenhar juntos o esqueleto de um dos programas mais icônicos na aprendizagem da programação. O famoso "Hello World" é como o primeiro "olá" de um bebê para o mundo, e no nosso caso, será o primeiro aceno para o universo do código. Vou guiá-los na escrita de um pseudocódigo que possa ser entendido como um convite amigável para quem está iniciando sua jornada em programação.

Então, sem mais delongas, abram suas mentes e preparem seus lápis (ou teclados), pois estamos prestes a decifrar juntos o código do pseudocódigo!

Introdução ao pseudocódigo.

O pseudocódigo é uma ferramenta fundamental no repertório de qualquer desenvolvedor, atuando como um intermediário linguístico entre a complexidade do pensamento humano e a precisão exigida pelas linguagens de programação. Tecnicamente falando, é uma maneira simplificada de representar os passos ou algoritmos que um programa irá executar, utilizando uma mistura de linguagem natural e uma estrutura que remete a elementos típicos de programação, sem aderir a nenhuma linguagem específica.

A utilização do pseudocódigo é um passo importante no processo de desenvolvimento de software, permitindo que algoritmos sejam pensados e discutidos de forma independente de linguagem. Isso traz uma série de benefícios práticos, tais como a facilitação da comunicação de ideias entre membros de uma equipe, a possibilidade de focar na lógica de programação sem a distração de sintaxes complexas, e a criação de um esboço que pode ser transformado em código em qualquer linguagem.

Um exemplo simples de pseudocódigo pode ilustrar seu uso. Se quisermos descrever um algoritmo para calcular o fatorial de um número, poderíamos escrever o seguinte:

```
algoritmo "CalculoFatorial"
var
    n, resultado, contador: inteiro

inicio
    escreva("Digite um numero inteiro positivo: ")
    leia(n)

    se n <= 1 entao
        resultado := 1
    senao
        resultado := n
        contador := n - 1
        enquanto contador > 1 faca
            resultado := resultado * contador
            contador := contador - 1
        fim-enquanto
    fim-se
    escreva("O fatorial de ", n, " é: ", resultado)
fimalgoritmo
```

Este pseudocódigo descreve claramente os passos lógicos necessários para calcular o fatorial de um número, sem que haja preocupação com a sintaxe específica de uma linguagem de programação. Ele é estruturado e suficientemente descritivo para permitir que qualquer desenvolvedor com conhecimento básico de programação possa transformá-lo em código.

Como veremos nos próximos capítulos, a estruturação do pseudocódigo pode variar, mas certos elementos são comuns, tais como declarações de variáveis, instruções condicionais (como "se-então-senão"), laços de repetição (como "enquanto" ou "para"), e a indicação clara de entradas e saídas do algoritmo.

A meta aqui é atingir um nível de abstração que promova a compreensão intuitiva dos procedimentos envolvidos, alinhando a solução proposta com a lógica do problema. Ao longo

deste capítulo, iremos solidificar nosso entendimento sobre esses conceitos e como aplicá-los eficientemente ao pseudocódigo para construir algoritmos robustos e claros.

Neste código, utilizamos a palavra-chave `var` para declarar variáveis. Solicitamos ao usuário que insira um valor para `n` utilizando `escreva` e `leia`. As operações condicionais e o loop são controlados por `se...entao...senao` e `enquanto...faca`, respectivamente. O resultado é atribuído usando `:=` e finalmente exibido com `escreva`.

Execute este programa no Visualg, e ele lhe pedirá um número inteiro. Após a entrada, o algoritmo calculará e exibirá o fatorial do número fornecido.

Sintaxe básica e estrutura.

Ao embarcarmos na jornada de aprendizagem do Visualg, é como se começássemos a nos familiarizar com a gramática de uma nova língua, só que muito mais divertido e interativo, pois estamos aprendendo a conversar com o computador! Vamos nos concentrar em compreender a sintaxe básica e a estrutura do Visualg, que facilitam o desenvolvimento de algoritmos, criando um paralelo com os elementos que já encontramos no mundo real.

Ao escrever um programa no Visualg, seguimos uma estrutura padronizada, que é quase como montar um quebra-cabeça: cada peça tem seu lugar e juntas elas formam a imagem que queremos visualizar. Abaixo estão os componentes essenciais que você precisa conhecer para começar a escrever seus próprios algoritmos no Visualg.

Declaração de Variáveis Pense nas variáveis como caixas de armazenamento, onde você pode guardar diferentes tipos de informações, como números ou textos. No Visualg, todas as variáveis devem ser declaradas no início do programa dentro do bloco `var`, determinando um nome e um tipo para cada uma.

Exemplo:

```
var
    numero: inteiro
    nome: caracter
    salario: real
```

Estrutura de um Algoritmo Um algoritmo no Visualg é sempre iniciado com a palavra-chave `algoritmo` seguida pelo nome do algoritmo e é concluído com `finalgoritmo`. Tudo o que está entre esses comandos é considerado parte do algoritmo.

Exemplo:

```
algoritmo "MeuPrimeiroAlgoritmo"
// Aqui vão as declarações de variáveis e a lógica do algoritmo
finalgoritmo
```

Entrada e Saída de Dados A interação com o usuário é feita através da entrada e saída de dados. Usamos `leia` para receber dados do usuário e `escreva` ou `escreval` para exibir mensagens ou resultados. O `escreval` é semelhante a `escreva`, mas ele adiciona uma quebra de linha automaticamente ao final.

Exemplo:

```
escreva("Informe seu nome: ")
leia(nome)
escreval("Olá, ", nome, "!")
```

Instruções Condicionais Assim como precisamos tomar decisões diárias baseadas em certas condições, os programas também precisam. No pseudocódigo do Visualg, utilizamos `se...entao...senao` para controlar o fluxo de um programa com base em condições.

Exemplo:

```
se idade >= 18 entao
    escreval("Você é maior de idade.")
senao
    escreval("Você é menor de idade.")
fim-se
```

Laços de Repetição Se você precisasse contar de 1 a 100, não faria sentido dizer cada número individualmente; você usaria uma forma de repetir o ato de contar. No Visualg, temos o enquanto...faca para repetir um bloco de código enquanto uma condição é verdadeira e para...ate...faca para repetições contadas.

Exemplo com enquanto:

```
contador := 1
enquanto contador <= 100 faca
    escreva(contador, " ")
    contador := contador + 1
fim-enquanto
```

Exemplo com para:

```
para contador de 1 ate 100 faca
    escreva(contador, " ")
fim-para
```

Comentários Comentários são notas que adicionamos ao código para explicar o que estamos fazendo, e no Visualg, começam com //. Eles são ignorados pelo programa e servem para orientar quem está lendo o código.

Exemplo:

```
// Isto é um comentário e não será processado pelo Visualg
```

Esses conceitos formam a base do que você precisa para começar a traduzir suas soluções em pseudocódigo utilizando o Visualg. Como em qualquer idioma, quanto mais você pratica, mais fluente se torna. Logo, você estará escrevendo algoritmos complexos com a naturalidade com a qual fala sua língua nativa!

Exemplo de pseudocódigo 1: "Hello World".

Imagine que ensinar o computador a dizer "Olá, Mundo!" é como ensinar uma criança a acenar. É uma tarefa simples, mas representa a primeira comunicação com o mundo ao redor.

Vou lhe mostrar como o "Olá, Mundo!" seria escrito em um pseudocódigo usando o Visualg:

```
algoritmo "OlaMundo"
inicio
    escreval("Olá, Mundo!") // Exibe a mensagem na tela
finalgoritmo
```

No exemplo acima, estamos seguindo as convenções que discutimos anteriormente. O programa começa com a palavra-chave `algoritmo`, seguida pelo nome do algoritmo, que nesse caso é "OlaMundo". Após isso, temos o bloco de código principal, delimitado pelas palavras `inicio` e `finalgoritmo`.

Dentro desse bloco, utilizamos `escreval` para imprimir na tela a frase "Olá, Mundo!". Como mencionamos antes, `escreval` adiciona automaticamente uma nova linha após a mensagem, o que é útil quando queremos que a próxima saída comece em uma nova linha.

Após escrever esse código no editor do Visualg, execute o programa e você verá a clássica saudação aparecer. E assim, com um simples gesto, seu algoritmo acenou para o mundo da programação, e você começou uma incrível jornada de descobertas!

Muito bem! É uma ótima ideia traçar paralelos entre pseudocódigo e linguagens de programação reais. O "Hello World!" é um ótimo exemplo para começar, já que ele é tradicionalmente o primeiro programa que se escreve quando se está aprendendo uma nova linguagem de programação.

A ideia por trás do "Hello World!" é simples: fazer com que o computador exiba essa saudação na tela. É como ensinar uma criança a dizer "olá" pela primeira vez. Uma vez que alcançamos esse pequeno sucesso, estamos prontos para nos aprofundar em conceitos mais complexos.

1. Pseudocódigo O pseudocódigo não é uma linguagem de programação verdadeira; em vez disso, é uma ferramenta de alto nível que permite aos programadores pensar sobre algoritmos de maneira estruturada, mas ainda em termos de linguagem natural. Para um "Hello World!", um pseudocódigo poderia ser tão simples quanto:

Início

Exibir "Hello, World!"

Fim

É uma representação genérica do que queremos que o programa faça, sem se preocupar com as sintaxes específicas de qualquer linguagem.

2. C C é uma linguagem de programação compilada que oferece um grande controle sobre o hardware, muito usada para sistemas e aplicativos onde o desempenho é crítico. Para escrever um "Hello World!" em C, o código se pareceria com isso:

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Aqui, estamos usando a função printf da biblioteca padrão de I/O para exibir a mensagem. O \n ao final da string é um caractere de nova linha, que faz com que o cursor se mova para o início da próxima linha após a mensagem ser exibida.

3. Python Python é uma linguagem de alto nível, interpretada e muito apreciada por sua simplicidade e legibilidade. Um "Hello World!" em Python é bastante direto:

```
print("Hello, World!")
```

Muito simples, não é? A função `print` cuida de tudo. Não precisamos importar bibliotecas adicionais ou definir uma função `main` especificamente — tudo faz parte do 'pacote básico' que Python fornece.

Esses exemplos mostram como diferentes linguagens atingem o mesmo objetivo, mas com diferentes níveis de complexidade e estrutura. O pseudocódigo nos ajuda a imaginar a solução, C nos dá controle e precisão, e Python nos oferece simplicidade e facilidade de uso. É como comparar uma receita escrita em linguagem simples para fazer um bolo, com as especificações detalhadas necessárias para um chef profissional e, finalmente, com um kit de bolo pronto para assar. Cada um tem seu lugar e valor no mundo da programação!

Vamos praticar!

Calculadora de IMC (Índice de Massa Corporal)

O IMC é uma medida de cálculo que utiliza a altura e o peso de uma pessoa. Vamos calcular o IMC e categorizar o resultado.

C:

```
#include <stdio.h>

int main() {
    float peso, altura, imc;

    printf("Insira seu peso em kg: ");
    scanf("%f", &peso);
    printf("Insira sua altura em metros: ");
    scanf("%f", &altura);

    imc = peso / (altura * altura);

    printf("Seu IMC é: %.2f\n", imc);

    if (imc < 18.5) {
        printf("Abaixo do peso.\n");
    } else if (imc >= 18.5 && imc < 25) {
        printf("Peso normal.\n");
    } else if (imc >= 25 && imc < 30) {
        printf("Sobrepeso.\n");
    } else {
        printf("Obesidade.\n");
    }

    return 0;
}
```

Python

```
peso = float(input("Insira seu peso em kg: "))
altura = float(input("Insira sua altura em metros: "))

imc = peso / (altura ** 2)

print(f"Seu IMC é: {imc:.2f}")

if imc < 18.5:
    print("Abaixo do peso.")
elif 18.5 <= imc < 25:
    print("Peso normal.")
elif 25 <= imc < 30:
    print("Sobrepeso.")
else:
    print("Obesidade.")
```

Verificador de Palíndromo

Um palíndromo é uma palavra, frase ou qualquer outra sequência de caracteres que lê o mesmo para frente e para trás, ignorando pontuação, maiúsculas e espaços.

C:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main() {
    char texto[100], texto_reverso[100];
    int i, j = 0, tamanho;

    printf("Digite uma palavra ou frase: ");
    gets(texto);

    tamanho = strlen(texto);

    for (i = tamanho - 1; i >= 0; i--){
        if (isalnum(texto[i])) {
            texto_reverso[j++] = tolower(texto[i]);
        }
    }

    texto_reverso[j] = '\0';

    if (strcmp(strlwr(texto), texto_reverso) == 0) {
        printf("É um palíndromo.\n");
    } else {
        printf("Não é um palíndromo.\n");
    }

    return 0;
}
```

Python

```
texto = input("Digite uma palavra ou frase: ").lower()
texto_limpo = ''.join(char for char in texto if char.isalnum())
texto_reverso = texto_limpo[::-1]

if texto_limpo == texto_reverso:
    print("É um palíndromo.")
else:
    print("Não é um palíndromo.")
```

Estes exemplos mostram como a mesma tarefa pode ser realizada em linguagens diferentes, respeitando as particularidades de cada uma delas. Em C, nos preocupamos muito com a manipulação direta e a alocação de arrays. Em Python, por outro lado, aproveitamos o dinamismo e as funções integradas para produzir um código mais conciso e expressivo. Como se estivéssemos construindo dois modelos de casas: uma com tijolos (C) e outra com peças de LEGO (Python), ambas capazes de fornecer um lar aconchegante, ainda que os processos e ferramentas usados sejam diferentes.

Vídeos para te ajudar:

Preparando um ambiente em Python no seu computador

<https://youtu.be/aFSnhN6spKU>

Caso não queira montar nenhum ambiente, pode usar recursos na nuvem

Python: <https://colab.research.google.com/>

Python, Javascript, C e outros: <https://replit.com/>

Seu primeiro algoritmo com Python

<https://youtu.be/7l7OriBEirs>

Livros que te ajudarão:

Este livro está disponível e gratuito no Github!!!

<https://penseallen.github.io/PensePython2e/>

Atividades:

Bora, para praticar, faremos o seguinte, prepare o seu ambiente, seja ele com C ou Python, tanto faz, você pode escolher qual caminho trilhará! Eu, particularmente gosto muito de Python como primeira linguagem de programação devido sua simplicidade!

Você é programador júnior, seu SQUAD recebeu a seguinte história de usuário:

Como, professor, quero que o sistema receba as notas de um aluno e faça o cálculo da média para eu ver se dou um décimozinho a mais naquela forcinha:

Critérios de aceitação:

1. O software deve receber 4 notas, digitadas pelo usuário, cada uma correspondente a um trimestre
2. O cálculo da média deve ser realizado com média aritmética simples, ou seja, soma as quatro notas e divide por 4
3. O resultado deve ser impresso na tela

Capítulo 4: Estruturas de Controle

Imagine que você está criando a coreografia de uma dança. Há momentos em que cada passo segue naturalmente o anterior, como em uma sequência. Porém, às vezes, é necessário decidir entre um passo ou outro com base na música que está tocando ou na reação do público. Além disso, certos movimentos precisam ser repetidos, criando um padrão rítmico que embeleza a performance. A construção de algoritmos e programas de computador não é muito diferente. Da mesma forma que um dançarino precisa de flexibilidade e controle para mudar ou repetir seus movimentos, precisamos de estruturas em nossa dança algorítmica que nos permitam controlar o fluxo de execução do programa.

As Estruturas de Controle são como os movimentos dessa dança que guiam a execução de um programa. Elas são a espinha dorsal que permite aos programas serem mais do que apenas uma lista de comandos executados de cima para baixo. Com estruturas de controle, podemos tomar decisões (usando estruturas condicionais), realizar ações repetidamente (por meio de estruturas de loop), e até mesmo interromper ou pular parte da coreografia quando for necessário (utilizando comandos como `break` e `continue`).

Neste capítulo, vamos explorar os seguintes conceitos e elementos que controlam o fluxo de um programa:

Estruturas Condicionais: Aprenderemos a usar `if`, `else if`, e `else`, que são como os passos de dança que dependem de certas notas musicais para acontecer. Eles nos permitem executar diferentes ações dependendo das condições encontradas durante a execução.

Estruturas de Loop: Descobriremos como as estruturas `for` e `while` podem nos ajudar a repetir um conjunto de movimentos, ou em termos de programação, a executar um bloco de código repetidamente até que uma determinada condição seja atendida ou deixe de ser atendida.

Desvios de Fluxo: Veremos como break, continue, e return atuam como os saltos e giradas repentinas que can change a 'melodia' de um programa, permitindo-nos sair de um loop ou ignorar uma iteração, ou mesmo retornar de uma função a qualquer momento.

Ao longo deste capítulo, faremos uma viagem detalhada através de cada um desses elementos, analisando exemplos e praticando com exercícios, para que você possa aprender a coreografia completa da programação e se tornar um verdadeiro maestro do código, capaz de orquestrar seu fluxo com confiança e criatividade.

Preparados? Então, vamos colocar a música para tocar e começar!

IF e ELSE.

A dança da programação está prestes a começar, e em nossa coreografia, as primeiras movimentações que vamos explorar são o if e else. Vamos imaginar que essas são as decisões que um dançarino faz para escolher entre um passo ou outro, dependendo da melodia que está sendo tocada.

No mundo da programação, if e else são usados para testar condições. Se a condição for verdadeira, então o programa executa um conjunto de instruções. Caso contrário, ele pode pular esse conjunto ou executar um conjunto alternativo de instruções.

Vamos a um exemplo bem simples: decidir se é um dia de semana ou fim de semana.

Suponha que em nosso programa de dança temos uma variável chamada dia que contém o dia da semana. Se for "sábado" ou "domingo", é dia de relaxar; caso contrário, é dia de praticar nossa dança, isto é, dia de semana comum de trabalho ou estudo.

Em C:

```
#include <stdio.h>
#include <string.h>

int main() {
    char dia[10];

    printf("Digite o dia da semana: ");
    scanf("%s", dia);

    if (strcmp(dia, "sabado") == 0 || strcmp(dia, "domingo") == 0) {
        printf("Hora de relaxar, é fim de semana!\n");
    } else {
        printf("Hora de praticar a dança da programação, é dia de semana.\n");
    }

    return 0;
}
```

No exemplo, a função `strcmp` é usada para comparar strings em C, já que não podemos comparar diretamente strings com o operador de igualdade `==` como fazemos com números.

Em Python:

```
dia = input("Digite o dia da semana: ").lower()

if dia == "sabado" or dia == "domingo":
    print("Hora de relaxar, é fim de semana!")
else:
    print("Hora de praticar a dança da programação, é dia de semana.")
```

Python torna a comparação de strings direta e intuitiva, sem a necessidade de funções auxiliares como em C. Além disso, utilizamos o método `lower()` para garantir que a comparação não seja afetada por maiúsculas ou minúsculas.

Nos dois exemplos, os programas pedem ao usuário para digitar o dia da semana e, em seguida, usam `if` para verificar se o dia digitado é sábado ou domingo. Se for um desses dias, eles executam um conjunto de instruções (relaxar). Se não, eles executam outro conjunto de instruções (praticar a dança da programação).

Assim como um dançarino observa seu parceiro ou a música para decidir seu próximo passo, nosso programa usa estruturas if e else para decidir qual código executar. É uma coreografia onde cada movimento é deliberado e cada passo tem propósito, permitindo criar uma performance dinâmica que responde ao ritmo do dia-a-dia do usuário.

Switch-Case

Na grande apresentação da programação, às vezes os dançarinos têm vários passos prontos para corresponder a diferentes partes da música. Imagine que temos uma orquestra tocando e dependendo do instrumento destacado, o dançarino saberá exatamente qual passo realizar. Na programação, podemos usar switch em C para escolher entre múltiplos caminhos com base em uma variável. Para Python, embora não haja um switch nativo, ainda podemos dirigir nosso dançarino com criatividade e estilo.

Exemplo com switch em C:

Imaginemos um programa para escolher a música de uma apresentação de dança baseada no humor do usuário.

```
#include <stdio.h>

int main() {
    int humor;

    printf("Como está se sentindo hoje? (1- Feliz, 2- Triste, 3- Animado, 4- Relaxado): ");
    scanf("%d", &humor);

    switch (humor) {
        case 1:
            printf("Tocando 'Happy' de Pharrell Williams.\n");
            break;
        case 2:
            printf("Tocando 'Someone Like You' de Adele.\n");
            break;
        case 3:
            printf("Tocando 'Uptown Funk' de Bruno Mars.\n");
            break;
        case 4:
            printf("Tocando 'Weightless' por Marconi Union.\n");
            break;
        default:
            printf("Humor não reconhecido, tocando 'Don't Worry Be Happy' de Bobby McFerrin.\n");
            break;
    }

    return 0;
}
```

Alternativa simulando switch em Python:

Em Python, vamos usar um dicionário e mapear os humores para funções que imprimem as mensagens correspondentes.

```

def happy():
    print("Tocando 'Happy' de Pharrell Williams.")

def sad():
    print("Tocando 'Someone Like You' de Adele.")

def excited():
    print("Tocando 'Uptown Funk' de Bruno Mars.")

def relaxed():
    print("Tocando 'Weightless' por Marconi Union.")

def default():
    print("Humor não reconhecido, tocando 'Don't Worry Be Happy' de Bobby McFerrin.")

humor = int(input("Como está se sentindo hoje? (1- Feliz, 2- Triste, 3- Animado, 4- Relaxado): "))

# Usando um dicionário como switch alternativo
switcher = {
    1: happy,
    2: sad,
    3: excited,
    4: relaxed
}

# Executando a função correspondente ao humor, ou default se não houver correspondência
switcher.get(humor, default)()

```

Nesses exemplos, estamos fazendo a coreografia do nosso programa se adaptar ao humor do usuário, dançando ao ritmo da emoção escolhida. Seja na sintaxe estrita de C com `switch`, ou na flexibilidade de Python com dicionários e funções, estamos criando um espetáculo de código que se altera para atender o público. Em termos de dança, `switch` é como uma série de passos que um dançarino memoriza e executa de acordo com o sinal diz - o líder da dança. No caso de Python, é como se o dançarino consultasse uma folha de dicas e improvisasse a partir das opções disponíveis para corresponder à música.

Laços de repetição: for, while, e do-while.

Vamos lá! Em nossa coreografia de código, chegamos ao momento dinâmico e energético: as estruturas de repetição. Assim como na dança, onde certos passos ou conjuntos de passos podem ser repetidos para criar padrões e ritmos, na programação também utilizamos estruturas como for, while e do-while para repetir blocos de código de forma controlada.

Dentro do ritmo pulsante da programação, as estruturas de repetição são como sequências de passos de dança que são realizados repetidamente até que uma certa condição de parada seja atingida.

O for: O loop for é como uma coreografia planejada. Você sabe exatamente quantos passos serão realizados antes de começar a dançar. No loop for, definimos uma variável de controle, uma condição de continuação e uma instrução de incremento ou decremento.

Exemplo em C:

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 5; i++) {
        printf("Repetição %d da dança\n", i);
    }
    return 0;
}
```

Exemplo em Python:

```
for i in range(1, 6):
    print(f"Repetição {i} da dança")
```

O while: O loop while é como improvisar na pista de dança sem planejamento prévio. Você dança enquanto está ouvindo a música; quando ela pára, você também pára. No loop while, a execução continua até que a condição especificada deixe de ser verdadeira.

```
#include <stdio.h>

int main() {
    int i = 1;
    while (i <= 5) {
        printf("Repetição %d da dança\n", i);
        i++;
    }
    return 0;
}
```

Agora em Python

```
i = 1
while i <= 5:
    print(f"Repetição {i} da dança")
    i += 1
```

O do-while: Este é um pouco diferente – é como dizer, "Vou dançar pelo menos uma vez, mesmo se a música já estiver acabando". O loop do-while garante que o bloco de código seja executado pelo menos uma vez antes de verificar a condição.

Exemplo em C:

```
#include <stdio.h>

int main() {
    int i = 1;
    do {
        printf("Repetição %d da dança\n", i);
        i++;
    } while (i <= 5);
    return 0;
}
```

Em Python, não existe uma estrutura do-while nativa, mas podemos emular seu comportamento com um loop while e uma condição verdadeira, como mostrado abaixo:

Alternativa para do-while em Python:

```
i = 1
while True:
    print(f"Repetição {i} da dança")
    i += 1
    if i > 5:
        break
```

Utilizando essas estruturas de repetição, podemos criar programas que executam tarefas repetidas vezes, como um dançarino que repete os mesmos movimentos em um determinado ritmo até que a música ou o coreógrafo indiquem a hora de parar. Cada estrutura tem seu papel, seu tempo, e sua elegância na dança do código. Com `for`, `while`, e `do-while`, os programas podem repetir seus passos graciosamente, garantindo que a performance seja tanto eficiente quanto encantadora.

Atividade Problema:

Imagine que você foi encarregado de organizar uma competição de dança – uma verdadeira maratona de dança! A competição durará várias horas, e para manter as coisas interessantes, você quer incluir uma variedade de estilos de dança. Os competidores deverão dançar consecutivamente uma série de músicas, cada uma representando um estilo diferente. No final de cada música, os juízes darão sua pontuação. A competição acaba quando uma dupla alcança uma pontuação total acumulada que ultrapasse um certo limite, demonstrando consistência e habilidade superiores.

Aqui está o desafio: Você precisa criar um programa que simule essa competição. A cada rodada (música), o programa irá receber as pontuações dos juízes para cada par de dançarinos e acumulará as pontuações. O programa deve continuar solicitando as pontuações da próxima rodada até que ao menos um par de dançarinos atinja a pontuação total necessária para ganhar. Quando isso acontecer, o programa deve anunciar o par vencedor e encerrar a competição.

O problema toca em diversos conceitos importantes, como:

Captura de entrada: O programa deve ser capaz de aceitar entradas para várias rodadas, simulando a continuidade da competição de dança.

Estruturas de Controle: Você precisará de estruturas de decisão para verificar as pontuações e determinar se os competidores atingiram a pontuação mínima para vencer.

Estruturas de repetição: Loops serão necessários para processar várias rondas de competição até que um vencedor seja declarado.

Resolução em C

```
#include <stdio.h>

int main() {
    int pontuacao, pontuacao_total = 0;
    int rodada = 1, limite_pontuacao = 100; // Definimos o limite de pontuação para
ganhar
    int continuar = 1;

    printf("Bem-vindos à Maratona de Dança!\n");
    while (continuar) {
        printf("Rodada %d\n", rodada);
        printf("Por favor, informe a pontuação dos juízes: ");
        scanf("%d", &pontuacao);

        pontuacao_total += pontuacao;

        if (pontuacao_total >= limite_pontuacao) {
            printf("Temos um vencedor! Parabéns aos dançarinos!\n");
            continuar = 0; // Encerra o loop
        } else {
            rodada++;
            printf("Pontuação atual: %d. Próxima rodada!\n", pontuacao_total);
        }
    }
    return 0;
}
```

Resolução em Python

```
pontuacao_total = 0
rodada = 1
limite_pontuacao = 100 # Definimos o limite de pontuação para ganhar

print("Bem-vindos à Maratona de Dança!")

while True:
    print(f"Rodada {rodada}")
    pontuacao = int(input("Por favor, informe a pontuação dos juízes: "))

    pontuacao_total += pontuacao

    if pontuacao_total >= limite_pontuacao:
        print("Temos um vencedor! Parabéns aos dançarinos!")
        break
    else:
        rodada += 1
        print(f"Pontuação atual: {pontuacao_total}. Próxima rodada!")
```

Desafio: Sistema Automatizado de Irrigação de Jardins

Você está ajudando a desenvolver um sistema inteligente de irrigação para um jardim botânico. Este sistema deve monitorar as condições de umidade do solo para cada tipo de planta e ajustar a quantidade de água fornecida automaticamente. Existem diferentes zonas no jardim, cada uma com um tipo de planta que possui requisitos específicos de umidade. O sistema de irrigação funcionará em ciclos, verificando a umidade do solo de cada zona e, se necessário, ativando a irrigação. O ciclo de checagem de umidade e irrigação continua até que todas as zonas estejam adequadamente úmidas. A tarefa é escrever um programa que simule o algoritmo de controle desse sistema de irrigação.

Conceitos envolvidos:

Leitura de Sensores: Representada pela entrada de dados simulando as leituras de umidade do solo.

Estruturas de Decisão: Decidir se uma zona precisa de irrigação com base na umidade do solo.

Estruturas de Repetição: Continuar o ciclo de verificação e irrigação até que todas as zonas atinjam a umidade desejada.

Capítulo 5: Estruturas de Dados Simples

Variáveis e tipos de dados (int, float, string, boolean).

Vetores/Arrays.

Exemplos de declaração e uso.

Exercício: Peça ao leitor para criar um pseudocódigo que armazene e imprima uma lista de compras.

Capítulo 6: Funções e Procedimentos

Definição de funções.

Vantagens do uso de funções em algoritmos.

Exemplo: função para calcular o quadrado de um número.

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- Função Quadrado(numero)
- Retorne numero * numero
- Fim Função
-
- Início
- Var resultado = Quadrado(5)
- Escrever resultado

Fim

Exercício: Desafie o leitor a criar uma função que calcule a média de três números.

Conclusão

Resumo do que foi discutido.

A importância de continuar praticando e estudando algoritmos e programação.

Próximos passos recomendados.

Referências

- Livros e recursos online recomendados para aprofundar conhecimentos.

Desafios e Exercícios Resolvidos

- Soluções comentadas dos exercícios propostos.